

# Network Traffic Forensic Analysis using Python

## Python Program:

```
with open("traffic.log","w") as f:
    f.write("10.0.0.9 -> 10.0.0.5:22 SYN\n")
    f.write("10.0.0.9 -> 10.0.0.5:23 SYN\n")
    f.write("10.0.0.9 -> 10.0.0.5:25 SYN\n")
    f.write("10.0.0.9 -> 10.0.0.5:80 SYN\n")
    f.write("10.0.0.9 -> 10.0.0.5:443 SYN\n")
    f.write("10.0.0.20 -> 10.0.0.5:80 SYN\n")
    f.write("10.0.0.20 -> 10.0.0.5:80 ACK\n")

ports_by_source = {}
with open("traffic.log") as f:
    for line in f:
        parts = line.split()
        src = parts[0]
        dst_port = parts[2].split(":")[1]
        ports_by_source.setdefault(src, set()).add(dst_port)

print("Network traffic forensic analysis:")
for src, ports in ports_by_source.items():
    print(f"{src}: contacted {len(ports)} distinct port(s)")
    if len(ports) >= 4:
        print("ALERT: Port-scanning behaviour detected")
```

## Sample Output:

```
Network traffic forensic analysis:
10.0.0.9: contacted 5 distinct port(s) -> ['22', '23', '25', '443', '80']
ALERT: 10.0.0.9 shows port-scanning behaviour
10.0.0.20: contacted 1 distinct port(s) -> ['80']
```

## Explanation:

This program demonstrates a simple network traffic forensic analysis. It reads a simulated network traffic log, counts the distinct destination ports contacted by each source IP address, and identifies systems that access many different ports. If a source IP contacts four or more distinct ports, it is flagged as a possible port-scanning attempt. This technique helps detect suspicious network activity during digital forensic investigations.